# Syllabus: From Ruby/Java Source to Running Linux Binary

**Target:** Programmer with strong application-layer knowledge, near-zero below-the-hood visibility.
**Priority:** Breadth → Speed → Practical → Deep
**Scope:** Linux x86-64, C family, stops at "compiled C binary running on OS"

---

## Module 1 — Execution Models (The Big Picture)

*Goal: Know what category every language falls into and why it matters.*

### Topics

- Interpretation vs. compilation (what actually happens at runtime)
- Bytecode: what it is, why it exists (portability, sandboxing)
- JIT compilation: tiered compilation, hotspot detection, deoptimization
- AOT compilation: trade-offs vs JIT
- Transpilation: when and why (CoffeeScript→JS, TypeScript→JS, Kotlin→JVM bytecode)
- Where Ruby MRI, JRuby, Rails, Java, Rust, and Python each sit on this map

### Key Resources

- Search for `CS:APP Bryant O'Hallaron 3rd edition` on the Web or Claude.ai — Ch. 1 (overview only)
- https://en.wikipedia.org/wiki/Just-in-time_compilation — read for breadth
- Search for `How the JVM JIT compiles code Cliff Click` on the Web or Claude.ai

### Practical

- Run `ruby --dump insns hello.rb` — see Ruby's bytecode
- Run `javap -c Hello.class` — read JVM bytecode for a hello world

---

## Module 2 — The C Compilation Pipeline

*Goal: Know every stage between a `.c` file and an executable.*

### Topics

1. **Preprocessing** (`cpp`): macros, `#include`, `#define`, conditional compilation
2. **Compilation** (`cc1`): C → Assembly (`.s`)

3. **Assembling** (`as`): Assembly → object file (`.o`) — ELF relocatable
4. **Linking** (`ld`): object files + libraries → ELF executable
5. **Loading** (`execve` + dynamic linker `ld-linux.so`): OS maps binary into memory

### Key Commands

```bash
gcc -E hello.c -o hello.i      # stop after preprocessing
gcc -S hello.c -o hello.s      # stop after compilation (see Assembly)
gcc -c hello.c -o hello.o      # stop after assembling
gcc   hello.c -o hello         # full pipeline
file hello                     # confirm ELF
```

### Key Resources

- Search for `CS:APP Bryant O'Hallaron 3rd edition` on the Web or Claude.ai — Ch. 7 (Linking)
- `man gcc`, `man ld`

---

## Module 3 — The ELF Binary Format

*Goal: Know what's inside a compiled binary before it runs.*

### Topics

- ELF header, program headers, section headers
- Key sections: `.text` (code), `.data` (initialized globals), `.bss` (zero-init), `.rodata`, `.plt`/`.got` (dynamic linking stubs)
- Symbol tables, relocation entries
- Static vs. dynamic linking — `.a` archives vs. `.so` shared objects
- `RPATH`, `LD_LIBRARY_PATH`, `ld-linux.so` resolution order

### Key Commands

```bash
readelf -a hello       # full ELF dump
objdump -d hello       # disassemble .text section
nm hello               # symbol table
ldd hello              # shared library dependencies
```

**Key Resources**

- Search for `Linkers and Loaders John Levine` on the Web or Claude.ai — Ch. 1–5
- `man elf`

---

## Module 4 — Assembly Fundamentals (x86-64, read-only goal)

*Goal: Read disassembly output. Not write Assembly.*

**Topics**

- Registers: `rax`, `rbx`, `rsp`, `rbp`, `rip`, `rflags` — what each is for
- Calling convention (System V AMD64 ABI): arguments in `rdi`, `rsi`, `rdx`... return in `rax`
- Stack frames: function prologue/epilogue, `push`/`pop`, `call`/`ret`
- Common instructions: `mov`, `lea`, `add`, `sub`, `cmp`, `jmp`, `jne`, `je`
- Reading a simple function's disassembly and mapping it back to C

**Key Resources**

- Search for `CS:APP Bryant O'Hallaron 3rd edition` on the Web or Claude.ai — Ch. 3 (Machine-Level Representation)
- https://godbolt.org — write C, watch Assembly live

**Practical**

- Take a 10-line C function, compile with `gcc -O0 -S`, read the output alongside CS:APP Ch. 3 (search above)

---

## Module 5 — How Linux Runs a Binary

*Goal: Know what happens between double-clicking (or `./hello`) and `main()` executing.*

**Topics**

- `execve()` syscall — how the kernel replaces a process image
- ELF program headers: `LOAD` segments mapped into virtual memory
- The dynamic linker (`ld-linux.so`) — runs before `main`, resolves shared libs
- `_start` → `__libc_start_main` → `main` — the real entry point chain

- Virtual address space layout: stack, heap, mmap regions, VDSO
- Syscalls: what they are, how `libc` wraps them (`write()` → `syscall` instruction)

### Key Commands

```bash
strace ./hello          # trace every syscall
/lib64/ld-linux-x86-64.so.2 --list ./hello   # run dynamic linker manually
cat /proc/<pid>/maps     # view process memory layout live
```

### Key Resources

- Search for `CS:APP Bryant O'Hallaron 3rd edition` on the Web or Claude.ai — Ch. 8 (Exceptional Control Flow)
- https://lwn.net/Articles/630727/ — "How programs get run" (part 1: execve mechanics)
- https://lwn.net/Articles/631631/ — "How programs get run: ELF binaries" (part 2: ELF loading)
- `man execve`, `man 5 proc`

---

## Module 6 — Debuggers at the Binary Level

*Goal: Use `gdb` to inspect a running process at the instruction level.*

### Topics

- `gdb` basics: `break`, `run`, `next`, `step`, `continue`, `quit`
- Binary-level commands: `disassemble`, `info registers`, `x/10i $rip`, `stepi`
- Inspecting the stack: `info frame`, `backtrace`, `x/20xg $rsp`
- Setting breakpoints on addresses, not just function names
- What DWARF debug info is — why `-g` flag changes what gdb can show you
- Debugging a stripped binary (no symbols) vs. unstripped

### Key Commands

```bash
```

```
gcc -g -o hello hello.c      # compile with debug info
gdb ./hello
(gdb) break main
(gdb) run
(gdb) disassemble
(gdb) stepi
(gdb) info registers
```

**Key Resources**

- Search for `CS:APP Bryant O'Hallaron 3rd edition` on the Web or Claude.ai — Ch. 3.10 (GDB usage)
- `gdb` built-in `help` command
- Search for `GDB is AWESOME Julia Evans` on the Web or Claude.ai

---

## Module 7 — Rust / LLVM Deep Dive (AOT, no runtime)

*Goal: Trace `.rs` → LLVM IR → Assembly → ELF. Understand the full compiler pipeline at depth.*

**Topics**

- `rustc` pipeline in full: parsing → HIR (High-level IR) → MIR (Mid-level IR) → LLVM IR → object file → linker
- HIR: desugared AST — where type inference and trait resolution happen
- MIR: Rust-specific CFG (control flow graph) — where borrow checking, lifetime analysis, and most optimizations happen
- LLVM IR: the portable intermediate representation — human-readable, typed, SSA form
- LLVM backend: IR → target-specific Assembly via instruction selection, register allocation, scheduling
- LLVM optimization passes: inlining, dead code elimination, loop unrolling (`-O0` vs `-O2` vs `-O3`)
- `rustc` produces standard ELF via `ld` (or `lld`) — no runtime, no GC, no JIT
- Monomorphization: generics are stamped out per concrete type at compile time → why Rust binaries can be large
- Zero-cost abstractions: what this means at the Assembly level — closures, iterators, etc.
- Comparing Rust vs C output for equivalent code on Godbolt — typically near-identical at `-O2`
- `#[no_std]` and bare-metal Rust: when there's no libc, no OS, no allocator

### Key Commands

```bash
rustc --emit=llvm-ir hello.rs          # LLVM IR (.ll file)
rustc --emit=asm hello.rs              # Assembly (.s file)
rustc --emit=mir hello.rs              # MIR dump
cargo rustc -- --emit=llvm-ir          # via cargo
cargo rustc -- -C opt-level=2 --emit=asm  # optimized Assembly
RUSTFLAGS="--emit=llvm-ir" cargo build    # project-wide
```

### Key Resources

- https://doc.rust-lang.org/nomicon/ — The Rustonomicon (official): unsafe Rust and compiler guarantees
- https://llvm.org/docs/LangRef.html — LLVM Language Reference Manual (reference, not cover-to-cover)
- Search for `Rust MIR Niko Matsakis 2016` on the Web or Claude.ai
- Search for `what does the Rust compiler do fasterthanlime` on the Web or Claude.ai
- https://godbolt.org — write Rust, compare LLVM IR and Assembly side by side

---

## Module 8 — JVM Essentials: How Java Differs from the C Path

*Goal: Understand what the JVM adds (and costs) vs. a native binary, without re-covering Module 7 ground.*

### Topics

- `javac`: Java → JVM bytecode (`.class` files) — portable, not native
- Why bytecode exists: write-once-run-anywhere, sandboxing, late binding
- Class loading: lazy, hierarchical (bootstrap → platform → app classloaders)
- Bytecode verification: the JVM's safety gate before any code runs
- Interpretation → tiered JIT (C1 → C2/HotSpot): how the JVM warms up
- Key contrast with Rust/C: no ELF produced at `javac` time — native code emitted at runtime by JIT
- GC and safepoints: why managed memory forces the JIT to insert yield points
- GraalVM Native Image: AOT-compiling Java to a real ELF — closes the gap with Rust/C
- JRuby: Ruby compiled to JVM bytecode — Module 8 applies fully

### Key Commands

```bash
javap -c -verbose Hello.class          # bytecode
java -XX:+PrintCompilation Hello       # watch JIT tier decisions
java -XX:+UnlockDiagnosticVMOptions \
    -XX:+PrintAssembly Hello           # JIT-emitted x86 (needs hsdis)
```

### Key Resources

- https://www.artima.com/insidejvm/ed2/ — *Inside the Java Virtual Machine* (Bill Venners), free online, Ch. 1–5
- https://wiki.openjdk.org/display/HotSpot/Main — OpenJDK HotSpot Internals wiki
- https://www.graalvm.org/latest/reference-manual/native-image/ — GraalVM Native Image overview

---

## Module 9 — Ruby MRI Essentials: Interpreter Path and Rails Context

*Goal: Understand how Ruby differs from both the C/Rust path and the JVM path, specifically in a Rails context.*

### Topics

- MRI Ruby (CRuby): the runtime is itself a C binary — Ruby executes inside a C process
- Parsing: `.rb` → AST → YARV bytecode (since Ruby 1.9) — never touches LLVM, never produces ELF
- YARV (Yet Another Ruby VM): stack-based bytecode interpreter — no JIT in MRI (YJIT added in 3.1, opt-in)
- Key contrast with JVM: no tiered JIT by default, no AOT option, slower warmup and steady-state
- YJIT (Ruby 3.1+): basic JIT — worth knowing exists but not deep-diving
- The GIL (Global VM Lock): one thread executes Ruby bytecode at a time — key architectural constraint
- C extensions (`.so` files): how gems like `nokogiri` or `pg` drop to native — bypasses GIL, native ELF
- JRuby as the alternative: compiles Ruby to JVM bytecode → Module 8 applies fully
- Rails startup sequence: `bundle exec rails server` → Bundler → `require` chain → Rack → Puma → your code
- What "slow Rails boot" actually is: thousands of `require` calls, each parsing and compiling `.rb` to YARV

### Key Commands

```bash
```

```
ruby --dump insns app.rb            # YARV bytecode
ruby --dump parsetree app.rb        # AST
RubyVM::InstructionSequence.compile(source).disasm   # from IRB
RUBY_YJIT_ENABLE=1 ruby app.rb      # enable YJIT
```

**Key Resources**

- Search for `Ruby Under a Microscope Pat Shaughnessy` on the Web or Claude.ai — the definitive book on MRI internals
- https://github.com/ruby/ruby/blob/master/insns.def — all YARV instructions defined in source
- Search for `YJIT Shopify engineering blog Ruby 3` on the Web or Claude.ai

---

## Module 10 — Reverse Engineering a C Binary

*Goal: Take a stripped ELF binary and recover intent.*

**Topics**

- Why reverse engineering: malware analysis, CTFs, closed-source debugging, auditing
- `objdump -d`: raw disassembly, no symbols
- `strings`: extract readable strings from binary
- `ltrace` / `strace`: trace library calls and syscalls at runtime
- Ghidra (NSA, free): decompiler — Assembly → pseudo-C
- Radare2/Cutter: open-source alternative
- Major hurdles:
  - Stripped symbols: no function names
  - Compiler optimizations: inlining, loop unrolling destroys structure
  - Obfuscation: intentional anti-analysis
  - Position-independent code (PIE) + ASLR: addresses change each run
- Best practices:
  - Start with `file`, `strings`, `ldd`, `strace` before disassembling
  - Identify `main` via `__libc_start_main` argument in `_start`
  - Rename functions as you understand them in Ghidra
  - Cross-reference strings → find where they're used → understand call graph

### Key Tools

```bash
file ./binary
strings ./binary | less
strace ./binary
ltrace ./binary
objdump -d ./binary | less
ghidra                  # GUI decompiler
```

### Key Resources

- Search for `Hacking The Art of Exploitation Erickson` on the Web or Claude.ai — practical binary internals
- https://github.com/NationalSecurityAgency/ghidra — Ghidra source, releases, and NSA training course
- https://picoctf.org — free RE challenges with progression (beginner-friendly)
- https://pwn.college — structured RE + binary exploitation curriculum

---

## Recommended Study Order & Time Estimate

| Module | Topic | Est. Hours |
|---|---|---|
| 1 | Execution Models | 3h |
| 2 | C Compilation Pipeline | 4h |
| 3 | ELF Binary Format | 3h |
| 4 | Assembly Fundamentals | 5h |
| 5 | Linux Binary Execution | 4h |
| 6 | Debuggers (binary level) | 4h |
| 7 | Rust / LLVM Deep Dive | 6h |
| 8 | JVM Essentials (Java/JRuby) | 3h |
| 9 | Ruby MRI / Rails Essentials | 3h |
| 10 | Reverse Engineering | 6h |
| **Total** | | **~41h** |

Modules 1–6 are the critical spine. Do them in order. Modules 7–10 are independent once 1–6 are done.

---

## Single Best Book

Search for `CS:APP Bryant O'Hallaron 3rd edition` on the Web or Claude.ai
Covers Modules 2, 3, 4, 5, 6 in one coherent arc. Get the 3rd edition.
If you only buy one book, this is it.

---

## Appendix — How This Syllabus Was Generated

This syllabus was produced by Claude (Sonnet 4.6) in an interactive session where the student answered a structured assessment. The Q&A below captures the full intake so a future LLM session can understand the student's baseline and the reasoning behind every scoping decision.

### Student Profile Assessment

**Q: Can you explain the difference between interpretation, bytecode compilation, JIT compilation, and AOT compilation without looking it up?**
A: Vaguely

**Q: How well do you understand what the JVM actually does at runtime (class loading, bytecode verification, JIT tiers, GC)?**
A: Surface — I know it's 'managed'

**Q: Rate your understanding of what a linker does (static vs dynamic linking, symbol resolution, ELF sections)**
A: 1–2: Heard the term

**Q: Have you ever read or written any Assembly, even toy examples?**
A: Seen it, didn't follow it

**Q: How familiar are you with what happens between `gcc hello.c` and a running process? (preprocessing, compiling, assembling, linking, loading)**
A: Just know it produces a binary

**Q: Have you used a debugger (gdb, lldb) at the binary/instruction level — not just line-by-line source debugging?**
A: Never used a debugger

**Q: Have you ever reverse engineered a compiled binary?**
A: Know what it is, never done it

**Q: How well do you understand how Linux loads and runs a binary? (ELF, syscalls, virtual memory, process space)**
A: Surface — I know Linux 'manages stuff'

**Q: What matters most to you in this self-study syllabus? (ranked)**
A: 1. Breadth (cover all topics), 2. Speed (learn it fast), 3. Practical skills (do it yourself), 4. Deep understanding (why it works)

**Derived Profile**

Strong at the application layer (Ruby/Rails, Java), near-zero visibility below it. No partial knowledge to patch — clean build from scratch. Priority ordering (breadth → speed → practical → deep) shaped module depth and ordering: Rust gets a full deep dive (Module 7) per student preference; JVM (Module 8) and Ruby MRI (Module 9) are scoped as "essentials" — enough to understand how they differ from the C/Rust path, not redundant deep dives. All external references are either direct verified URLs or explicit Search for ${TOPIC} instructions.